

SECURED MESSAGING EXCHANGE PROCEDURE

V 2.0

25th Mar 2021

DEFINITION OF TERMS

CSR – Certificate Signing Request

SHA – Secure Hash Algorithm

IPS – Instant Payment Switch

X.509 – A standard defining the format of public key certificates.

1. GENERATING CERTIFICATES FOR IPS

(i) DOWNLOAD AND SETUP OPENSSL

Visit below site to download OpenSSL

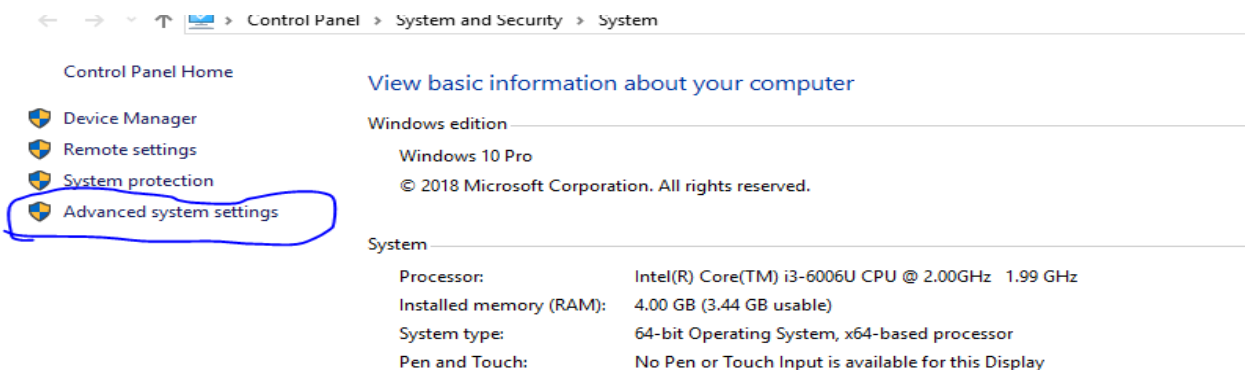
<https://slproweb.com/products/Win32OpenSSL.html>

Download Win32/Win64 OpenSSL today using the links below!

File	Type	Description
Win64 OpenSSL v1.1.1 Light EXE MSI	3MB Installer	Installs the most commonly used essentials of Win64 OpenSSL v1.1.1 (Recommended for users by the creators of OpenSSL). Only installs on 64-bit versions of Windows. Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win64 OpenSSL v1.1.1 EXE MSI	63MB Installer	Installs Win64 OpenSSL v1.1.1 (Recommended for software developers by the creators of OpenSSL). Only installs on 64-bit versions of Windows. Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win32 OpenSSL v1.1.1 Light EXE MSI	3MB Installer	Installs the most commonly used essentials of Win32 OpenSSL v1.1.1 (Only install this if you need 32-bit OpenSSL for Windows. Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win32 OpenSSL v1.1.1 EXE MSI	54MB Installer	Installs Win32 OpenSSL v1.1.1 (Only install this if you need 32-bit OpenSSL for Windows. Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.

Install following the default options

Go to window properties, Advanced Settings

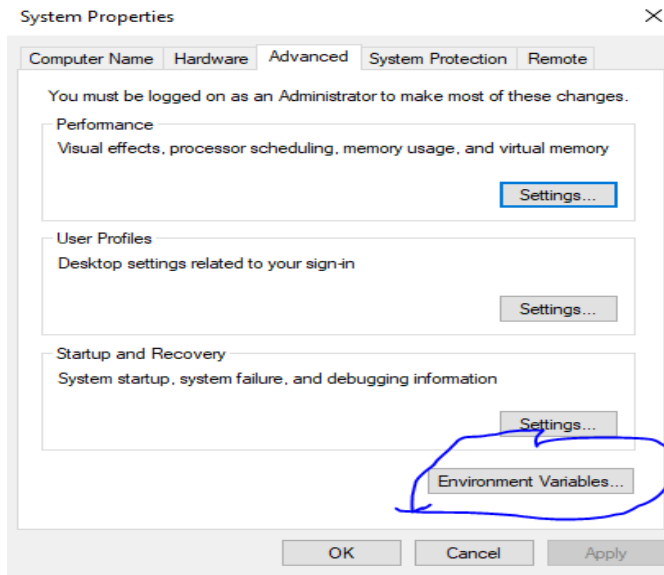


The screenshot shows the Windows Control Panel with the 'System' window open. In the left-hand navigation pane, 'Advanced system settings' is highlighted with a blue circle. The main content area displays 'View basic information about your computer' for Windows 10 Pro. Under the 'System' section, the following details are listed:

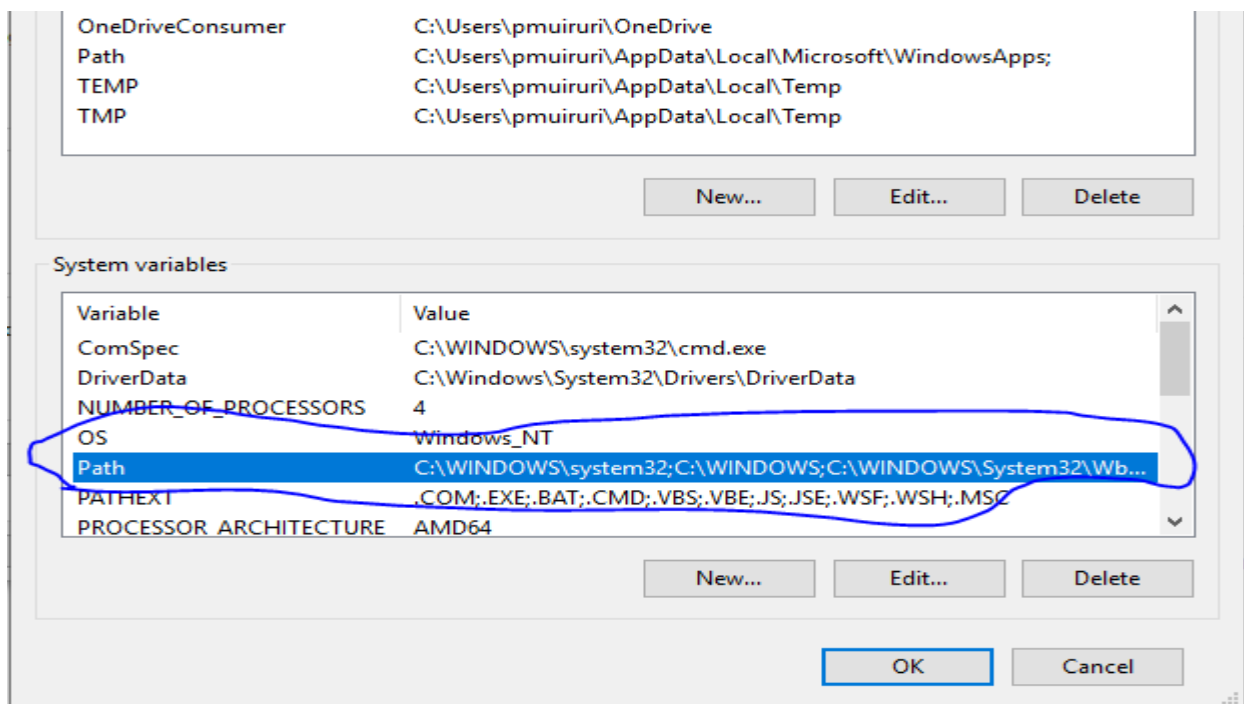
- Processor: Intel(R) Core(TM) i3-6006U CPU @ 2.00GHz 1.99 GHz
- Installed memory (RAM): 4.00 GB (3.44 GB usable)
- System type: 64-bit Operating System, x64-based processor
- Pen and Touch: No Pen or Touch Input is available for this Display

Click on **Environmental Variables**

Secured Messaging Exchange Procedure

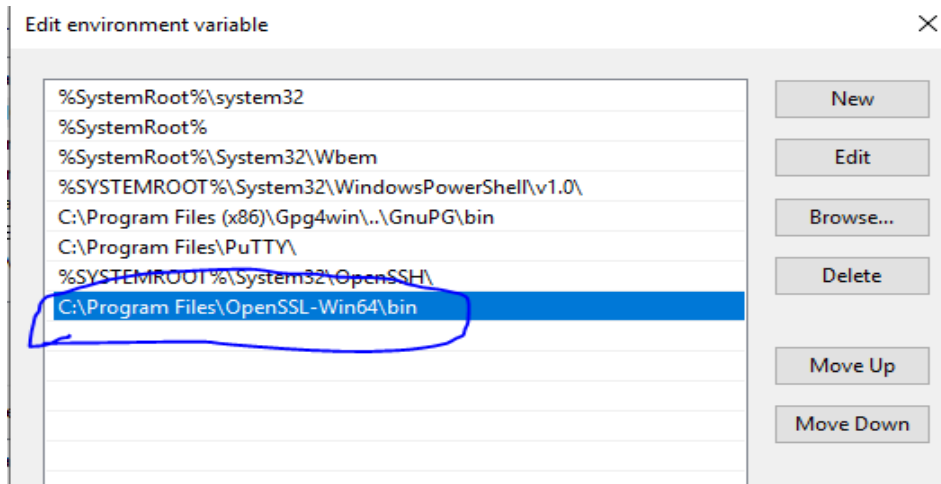


Double-click on "PATH"



If you don't have this path listed, click on NEW, add it and save

C:\Program Files\OpenSSL-Win64\bin



Now open CMD, type “openssl” under the folder you want to generate the certificates from.

```
C:\Users\pmuiruri\Documents\Certificates and keys>  
C:\Users\pmuiruri\Documents\Certificates and keys>  
C:\Users\pmuiruri\Documents\Certificates and keys>openssl  
OpenSSL>  
OpenSSL>
```

You can now proceed to generate the certificates.

(ii) IPSL Participant Bank Certificate Generation

The participant entities will require two x.509 certificates signed by IPSL's certificate authority for transport and message signing purposes.

IPSL's public API endpoint requires mutual TLS authentication, so the participant needs to supply a client SSL certificate when connecting to it.

The transport certificate is used for that purpose. Similarly, the participant bank also publishes an API callback endpoint on their side, which IPS API service connects to via HTTPS; the same transport certificate is used as the server HTTPS certificate on their side.

The signature certificate is used to sign the messages being sent to IPS API.

CSR generation

This process occurs at the participant's side. They generate two different sets of private keys and Certificate Signing Requests.

Example OpenSSL commands are shown below:

To generate Transport/ Client Key and CSR:

```
# openssl req -new -newkey rsa:2048 -nodes -subj "/C=KE/CN=Participant Bank  
Name Transport" -addext "subjectAltName = DNS:api.participant.co.ke" -keyout  
BANKXX_transport.key -out BANKXX_transport.csr
```

To generate Signing/ Seal Key and CSR:

```
# openssl req -new -newkey rsa:2048 -nodes -subj "/C=KE/CN=Participant Bank  
Name Signing" -addext "subjectAltName = DNS:api.participant.co.ke" -keyout  
BANKXX_seal.key -out BANKXX_seal.csr
```

```
C:\CA\ipsl>openssl req -new -newkey rsa:2048 -nodes -subj "/C=KE/CN=Participant  
Bank Name" -addext "subjectAltName = DNS:api.participant.co.ke" -keyout BANKXX  
_seal.key -out BANKXX_seal.csr  
Generating a RSA private key  
.....+++++  
....+++++  
writing new private key to 'BANKXX_seal.key'  
-----
```

The **subjectAltName** extension should contain the FQDN of the participant's own API endpoint; this needs to match in order to allow for successful connection from IPSL's API to the participant's servers.

The resulting **.key** files will contain private RSA keys that the participant must store in a secure manner and never share with anybody including IPSL. The **.csr** files are to be submitted to IPSL for signing.

At this point, you should have a private key and CSR created with names **BANKXX_seal.key** and **BANKXX_seal.csr** respectively.

Name	Date modified	Type	Size
 BANKXX_seal.csr	11-Feb-21 3:24 PM	CSR File	1 KB
 BANKXX_seal.key	11-Feb-21 3:02 PM	KEY File	2 KB

- You can verify the CSR with the command: **openssl req -text -in BANKXX_seal.csr -noout -verify**. With this, you can verify if the details you entered are correct. If not, repeat the procedure above to re-generate the CSR.

```
C:\CA\ipsl>openssl req -text -in BANKXX_seal.csr -noout -verify
verify OK
Certificate Request:
Data:
  Version: 1 (0x0)
  Subject: C = KE, CN = Participant Bank Name
  Subject Public Key Info:
    Public Key Algorithm: rsaEncryption
    RSA Public-Key: (2048 bit)
    Modulus:
      00:c0:df:cb:bc:99:37:a5:81:88:80:3a:10:76:26:
      25:76:76:5f:3f:f7:7e:c6:3a:6f:3b:01:1b:52:6a:
      3e:27:28:ba:28:49:af:44:42:eb:28:cb:8c:b1:d8:
      35:ed:cc:d4:fd:30:ce:71:af:a3:01:c4:ca:90:fd:
      8f:24:54:8b:6f:fd:03:31:05:1c:17:60:ef:6f:69:
      ed:91:d0:9c:19:a2:36:8d:9a:0c:89:d1:8e:78:47:
      9d:44:3c:b9:f1:f3:db:8e:4a:87:79:1a:43:23:24:
      2e:ab:dc:ad:45:90:24:3a:38:17:f0:e3:fc:af:00:
      d4:de:1f:21:28:1f:23:a7:e2:20:68:ac:22:48:df:
      d2:65:03:2e:00:75:36:aa:c4:b8:8a:50:b3:08:f1:
      e1:a5:ea:4b:96:15:de:1f:36:58:fc:d0:52:59:33:
      f3:50:6d:23:94:d6:16:b9:c3:46:f3:36:83:02:45:
      03:42:e5:fc:9d:e9:5d:f0:8d:25:46:06:5c:a8:b6:
      fb:1c:eb:6a:5f:b1:48:f7:91:75:fa:94:b5:6c:8e:
      11:88:14:7a:89:1a:c7:44:5f:ad:1f:9e:c3:aa:a6:
```

NB: While trying to verify the CSR, and you encounter an error like the one shown below, exit openssl, launch it again, then retry the process.

```
OpenSSL> req -text -in BANKXX_seal.csr -noout -verify
problem creating object tsa_policy1=1.2.3.4.1
6940:error:08064066:object identifier routines:OBJ_create:oid exists:crypto\objects\obj_dat.c:698:
error in req
OpenSSL> exit

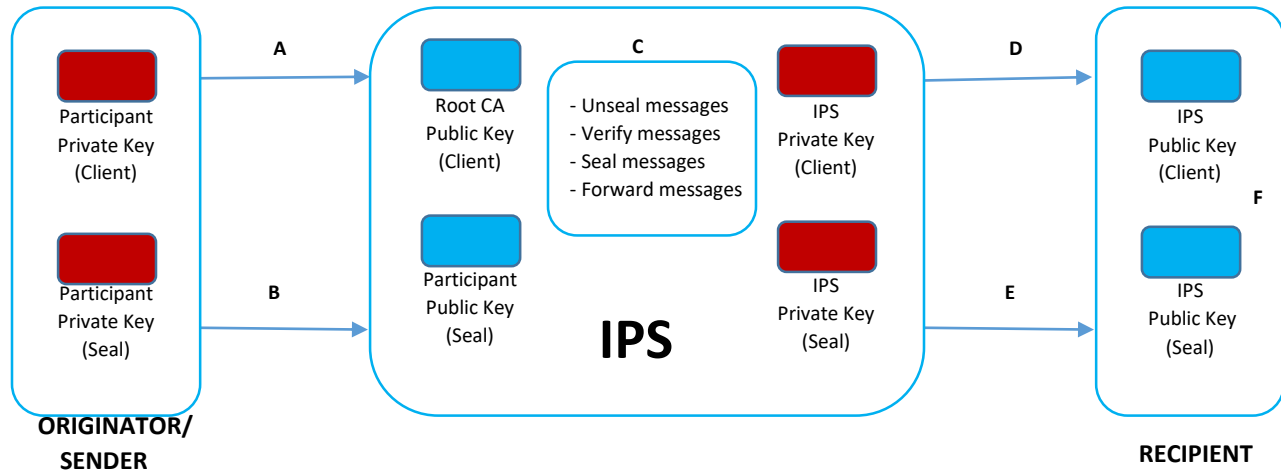
C:\Users\pmuiruri\Documents\Certificates and keys>openssl
OpenSSL> req -text -in BANKXX_seal.csr -noout -verify
verify OK
Certificate Request:
Data:
  Version: 1 (0x0)
  Subject: C = KE, ST = NAIROBI, L = NAIROBI, O = Internet Widgits Pty Ltd
  Subject Public Key Info:
    Public Key Algorithm: rsaEncryption
    RSA Public-Key: (2048 bit)
    Modulus:
```

Once you have generated the CSR, send the CSR to IPSL for signing. IPSL will sign the CSR, generate and send you back your x.509 certificate or public key.

IPSL will also share the root ca and intermediate certificates with all participants. Each participant will have to store these certificates in their Certificate Trust Store.

2. CERTIFICATE SHARING PROCEDURE

CERTIFICATES EXCHANGE DIAGRAM



- Client initiates a secure client connection between the sender and IPS using the participant's client private key. IPS verifies that this connection belongs to the purported user by verifying the client request against their stored public key.
- Once a secure connection has been established between the two parties, the sender will then digitally sign the message to be sent using their seal private key. The participant will then send the digitally signed message through this secure TLS connection.
- IPS will receive the message and decrypt the digital signature using the participant's public key. If the public key can't decrypt the digital signature it means the signature isn't the participant's and the message will be rejected. If decryption succeeds, it will then calculate the hash of the original message, then compare the calculated hash with the decrypted hash from original message. Any difference would reveal tampering of the message and would thus be declined.
- IPS initiates a secure client connection between the IPS and recipient using the IPS's client private key. Recipient verifies that this connection belongs to IPS user by verifying the client request against the *stored* IPS client public key.

Secured Messaging Exchange Procedure

- E. Once a secure connection has been established between the two parties, IPS will then digitally sign the message to be sent using IPS seal private key. IPS will then send the digitally signed message through this secure TLS connection.
- F. Recipient will receive the message and decrypt the digital signature using IPS's public key. If the public key can't decrypt the digital signature it means the signature isn't IPS's and the message will be rejected. If decryption succeeds, it will then calculate the hash of the original message, then compare the calculated hash with the decrypted hash from original message. Any difference would reveal tampering of the message and would thus be declined. If successful, the message is valid and processing continues.

This entire process can be summarized in the diagram below:

